



LI.FI

LI.FI Security Review

DeBridgeDInFacet.sol(v1.1.0)

Security Researcher

Sujith Somraaj (somraajsujith@gmail.com)

Report prepared by: Sujith Somraaj

April 30, 2026

Contents

1	About Researcher	2
2	Disclaimer	2
3	Scope	2
4	Risk classification	2
4.1	Impact	2
4.2	Likelihood	3
4.3	Action required for severity levels	3
5	Executive Summary	3
6	Findings	4
6.1	Low Risk	4
6.1.1	Validate orderAuthorityDst in DeBridgeDlnFacet	4
6.1.2	givePatchAuthoritySrc set to msg.sender strands DLN patch authority on Permit2Proxy when bridging via permit-based entry points	4
6.2	Informational	4
6.2.1	Missing documentation on DeBridge DLN auto-cancellation refund mechanics	4

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over \$170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast and Angles

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

- src/Facets/DeBridgeDInFacet.sol(v1.1.0)

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

- High** leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium** global losses <10% or losses to only a subset of users, but still unacceptable.
- Low** losses will be annoying but bearable — applies to things like grieving attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

High almost certain to happen, easy to perform, or not easy but highly incentivized

Medium only conditionally possible or incentivized, but still relatively likely

Low requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Critical Must fix as soon as possible (if already deployed)

High Must fix (before deployment if not already deployed)

Medium Should fix

Low Could fix

5 Executive Summary

Over the course of 3 hours in total, [LI.FI](#) engaged with the [researcher](#) to audit the contracts described in section 3 of this document ("scope").

In this period of time a total of 3 issues were found.

Project Summary	
Project Name	LI.FI
Repository	lifinance/contracts
Commit	3b8511fc
Audit Timeline	April 30, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found	
Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	2
Gas Optimizations	0
Informational	1
Total Issues	3

6 Findings

6.1 Low Risk

6.1.1 Validate orderAuthorityDst in DeBridgeDlnFacet

Context: [DeBridgeDlnFacet.sol#L190](#)

Description: Since allowedCancelBeneficiarySrc is hardcoded to empty strings in _startBridge() function, all cancel/refund flow depends on orderAuthorityAddressDst: only it can initiate cancellation, and it chooses the refund recipient at cancel time (per IDlnSource.sol:36-41).

The facet currently performs no validation on _deBridgeData.orderAuthorityDst. If empty or zero-address bytes are passed, the resulting order is uncancellable and unrefundable, permanently stranding funds if unfilled.

Recommendation: Consider adding the following defensive check inside the _startBridge() function:

```
+ if (_deBridgeData.orderAuthorityDst.length == 0)
+     revert InvalidOrderAuthorityDst();
```

LI.FI: Fixed in [36a8922](#)

Researcher: Verified fix.

6.1.2 givePatchAuthoritySrc set to msg.sender strands DLN patch authority on Permit2Proxy when bridging via permit-based entry points

Context: [DeBridgeDlnFacet.sol#L189](#)

Description: In _startBridge() function the DLN order is created with:

```
givePatchAuthoritySrc: msg.sender,
```

Per IDlnSource interface, givePatchAuthoritySrc is the address on the source chain authorised to patch the order by adding more input tokens, making it more attractive to solvers if it stalls.

When the facet is invoked through Permit2Proxy (callDiamondWithEIP2612Signature, callDiamondWithPermit2, callDiamondWithPermit2Witness, callDiamondWithPermit2SignatureSingle), the diamond is reached via _executeCalldata(), so msg.sender inside the facet is the Permit2Proxy contract address, not the user who signed the permit.

As a result, the DLN order's patch authority is assigned to Permit2Proxy, which exposes no function to call DLN's patch endpoint. The end user who is the economic owner of the order has no way to patch their own order.

This is the symmetric issue to the refund-routing problem fixed in this audit scope (allowedCancelBeneficiarySrc delegated to orderAuthorityAddressDst), where msg.sender-based authority assignment was likewise unsafe under the Permit2Proxy flow.

Recommendation: Stop deriving givePatchAuthoritySrc from msg.sender. Instead, source it from caller-supplied data so it remains correct whether the facet is invoked directly by an EOA or indirectly through Permit2Proxy.

LI.FI: Acknowledged. Having givePatchAuthoritySrc as msg.sender is not ideal, however user can always control the order from the destination chain via orderAuthorityAddressDst.

Researcher: Acknowledged.

6.2 Informational

6.2.1 Missing documentation on DeBridge DLN auto-cancellation refund mechanics

Context: [DeBridgeDlnFacet.sol#L19](#)

Description: DeBridgeDlnFacet relies on several DLN order parameters whose behaviour is not documented in this repo, but which directly govern whether and how a user can recover funds from an unfilled order.

The undocumented behaviours are:

1. Who can cancel and where refunds go. In `_startBridge()`, `allowedCancelBeneficiarySrc` is hardcoded to `""`, so cancellation can only be initiated by `orderAuthorityAddressDst`, and that same party chooses the source-chain refund recipient at cancel time. If `orderAuthorityDst` is misconfigured, the order becomes unrefundable.
2. Auto-cancellation feature. DeBridge can auto-cancel unfilled orders (5 min for unprofitable, 15 min for other causes) with subsidised gas, but only for market orders and only after integrator allow-listing. Nothing in the repo signals that this feature exists or that activation requires DeBridge approval.
3. Refund asset may differ from the input asset. DeBridge swaps inputs into reserve assets on order creation, so refunds are denominated in `srcChainTokenOut` (or `srcChainTokenIn` if absent), not necessarily the original token. Users and integrators are unlikely to expect this.

None of the above is captured in `docs/DeBridgeDlnFacet.md` or in `NatSpec`. Integrators and auditors have no in-repo reference for the cancellation, refund, and patch-authority model.

Recommendation: Add a Cancellation & Refund Semantics section to `docs/DeBridgeDlnFacet.md` covering points 1–3 above, plus a link to <https://docs.debridge.com/dln-details/affiliates/auto-cancellations> as the upstream reference.

Mirror the key trust assumption in inline documentation of the facet.

LI.FI: Fixed in [2535d74](#)

Researcher: Verified fix.